Hospital Database System Design and Implementation Report

Introduction

This report documents the design and implementation of a hospital database system developed for managing patients, doctors, appointments, departments, medical records, diagnoses, prescriptions and allergies. The database was designed and implemented using Microsoft SQL Server Management Studio (SSMS) and T-SQL.

The objective of the system is to provide a centralised solution for storing and managing hospital information while ensuring data integrity, reducing data redundancy and supporting efficient retrieval of information. The database was designed based on the requirements gathered during the initial consultation with the hospital.

The report explains the database design process, normalization, implementation of tables and constraints, creation of stored procedures, functions, views and triggers, and the queries required by the client. Screenshots of the database diagram, SQL code and outputs should be included throughout the report to support the explanations provided.

Part 1: Database Design and Normalisation

1. Database Design

The first stage of the project involved analysing the business requirements provided by the hospital and identifying the main entities required within the database.

The following entities were identified:

- Patient
- Doctor
- Department
- Appointment
- MedicalRecord
- Diagnosis
- Prescription
- PatientAllergy

Each entity was created as a separate table to ensure that data could be stored efficiently and relationships could be properly maintained.

Assumptions Made

The following assumptions were made during the design process:

1. A patient can have multiple appointments.
2. A doctor can attend multiple appointments.
3. A department can have multiple doctors.
4. Each appointment is associated with one doctor and one patient.
5. Each appointment can generate one medical record.
6. A medical record can contain multiple diagnoses.
7. A medical record can contain multiple prescriptions.
8. A patient can have multiple allergies.
9. Patient records should remain in the database even after leaving the hospital. The DateLeftHospital field was included to support this requirement.
10. Passwords should not be stored in plain text. Therefore, a PasswordHash field was used instead of storing actual passwords.

1.1 Normalisation Process

The database was normalised to Third Normal Form (3NF) to reduce redundancy and improve data consistency.

1NF

To satisfy First Normal Form, all tables were designed so that each column contains only atomic values and there are no repeating groups.

For example, allergies were not stored as a comma-separated list within the Patient table. Instead, a separate PatientAllergy table was created so that each allergy is stored as a separate record.

Similarly, diagnoses and prescriptions were separated from the MedicalRecord table because a patient may receive multiple diagnoses and medications during a visit.

2NF

Second Normal Form was achieved by ensuring that all non-key attributes are fully dependent on the entire primary key.

For example, the PatientAllergy table uses a composite primary key consisting of PatientID and AllergyName. Both attributes are required to uniquely identify a patient's allergy.

3NF

Third Normal Form was achieved by removing transitive dependencies.

Department information was separated into a dedicated Department table rather than storing department names directly within the Doctor table. This prevents duplication and ensures that department information is maintained in a single location.

Similarly, diagnosis and prescription information were separated into their own tables and linked through foreign keys to the MedicalRecord table.

1.2 Table Design and Relationships

Department Table

The Department table stores information about hospital departments.

Primary Key:

- DepartmentID

Important attributes:

- DepartmentName

The DepartmentID field was implemented as an IDENTITY column to automatically generate unique department identifiers.

Patient Table

The Patient table stores registration details for patients.

Primary Key:

- PatientID

Important attributes:

- FirstName
- LastName
- Address
- DateOfBirth
- Insurance
- Username
- PasswordHash
- Email
- PhoneNumber
- RegistrationDate
- DateLeftHospital

The PatientID field was implemented as an IDENTITY column to ensure uniqueness.

The PasswordHash field was used instead of storing passwords directly to improve security.

```
## 1.2 PATIENT TABLE

CREATE TABLE Patient (
PatientID INT IDENTITY(1,1) PRIMARY KEY,
FirstName VARCHAR(50) NOT NULL
CHECK (LEN(TRIM(FirstName)) > 0),
LastName VARCHAR(50) NOT NULL
CHECK (LEN(TRIM(LastName)) > 0),
Address VARCHAR(150) NOT NULL
CHECK (LEN(TRIM(Address)) > 0),
DateOfBirth DATE NOT NULL
CHECK (DateOfBirth <= CAST(GETDATE() AS DATE)),
Insurance VARCHAR(50) NOT NULL
CHECK (LEN(TRIM(Insurance)) > 0),
Username VARCHAR(50) NOT NULL UNIQUE
CHECK (LEN(TRIM(Username)) > 0),
PasswordHash VARCHAR(255) NOT NULL
CHECK (LEN(TRIM(PasswordHash)) > 0),
Email VARCHAR(100),
PhoneNumber VARCHAR(20),
RegistrationDate DATE NOT NULL
CHECK (RegistrationDate <= CAST(GETDATE() AS DATE)),
DateLeftHospital DATE,
CHECK (DateLeftHospital >= RegistrationDate)
);
```

	PatientID	FirstName	LastName	Address	DateOfBirth	Insurance	Username	Email	PhoneNumber	RegistrationDate	DateLeftHospital	PasswordHash
1	1	Alice	Walker	12 Main Street	1990-05-15	AXA	alice90	alice@email.com	1234567890	2025-01-01	NULL	HASH001
2	2	Robert	King	15 Oak Road	1985-08-22	Allianz	rob85	robert@email.com	1234567891	2025-01-05	NULL	HASH002
3	3	Linda	Scott	20 Pine Lane	1995-03-10	AXA	linda95	linda@email.com	1234567892	2025-01-10	NULL	HASH003
4	4	Mark	Evans	7 River Ave	1988-07-18	Bupa	mark88	mark@email.com	1234567893	2025-01-12	NULL	HASH004
5	5	Nancy	Young	40 Green St	1992-12-01	Allianz	nancy92	nancy@email.com	1234567894	2025-01-15	NULL	HASH005
6	6	Peter	Hall	55 King Rd	1980-06-06	Bupa	peter80	peter@email.com	1234567895	2025-01-20	NULL	HASH006
7	7	Olivia	White	3 Queen Ave	2000-09-09	AXA	olivia00	olivia@email.com	1234567896	2025-01-25	NULL	HASH007

Doctor Table

The Doctor table stores doctor information.

Primary Key:

- DoctorID

Foreign Key:

- DepartmentID

Each doctor belongs to one department, while a department can contain multiple doctors.

	DoctorID	FirstName	LastName	Specialty	DepartmentID
1	1	Ugo	Ikem	General Surgeon	7
2	2	Sarah	Johnson	Neurologist	2
3	3	Michael	Brown	Orthopedic Surgeon	3
4	4	Emma	Davis	Pediatrician	4
5	5	David	Wilson	Dermatologist	5
6	6	Sophia	Taylor	Oncologist	6
7	7	James	Anderson	General Surgeon	7

Appointment Table

The Appointment table stores patient appointments.

Primary Key:

- AppointmentID

Foreign Keys:

- PatientID
- DoctorID

Constraints were implemented to ensure:

- Appointment dates cannot be in the past.
- Status values are restricted to valid options.
- Doctors cannot be double booked.

- Patients cannot create duplicate appointments at the same date and time.

```
## 1.4 APPOINTMENT TABLE

CREATE TABLE Appointment (
AppointmentID INT IDENTITY(1,1) PRIMARY KEY,
AppointmentDate DATE NOT NULL
CHECK (AppointmentDate >= CAST(GETDATE() AS DATE)),
AppointmentTime TIME NOT NULL,
Status VARCHAR(20) NOT NULL
CHECK (Status IN ('Pending', 'Cancelled', 'Completed', 'Available')),
Feedback VARCHAR(500),
ArrivalTime TIME,
CompletedTime TIME,
CHECK (CompletedTime > ArrivalTime),
PatientID INT NOT NULL,
FOREIGN KEY (PatientID)
REFERENCES Patient(PatientID),
DoctorID INT NOT NULL,
FOREIGN KEY (DoctorID)
REFERENCES Doctor(DoctorID),
UNIQUE (DoctorID, AppointmentDate, AppointmentTime),
UNIQUE (PatientID, AppointmentDate, AppointmentTime)
);
```

Results Messages

	AppointmentID	AppointmentDate	AppointmentTime	Status	Feedback	ArrivalTime	CompletedTime	PatientID	DoctorID
1	1	2026-07-01	09:00:00.0000000	Pending	NULL	NULL	NULL	1	1
2	2	2026-07-01	10:00:00.0000000	Pending	NULL	NULL	NULL	2	1
3	3	2026-07-02	11:00:00.0000000	Pending	NULL	NULL	NULL	3	2
4	4	2026-07-02	12:00:00.0000000	Pending	NULL	NULL	NULL	4	3
5	5	2026-07-03	09:30:00.0000000	Pending	NULL	NULL	NULL	5	4
6	6	2026-07-03	10:30:00.0000000	Pending	NULL	NULL	NULL	6	5
7	7	2026-07-04	14:00:00.0000000	Pending	NULL	NULL	NULL	7	6

MedicalRecord Table

The MedicalRecord table stores records created from appointments.

Primary Key:

- MedicalRecordID

Foreign Key:

- AppointmentID

A unique constraint was applied to AppointmentID to ensure that each appointment can generate only one medical record.

Diagnosis Table

The Diagnosis table stores diagnoses associated with a medical record.

Primary Key:

- DiagnosisID

Foreign Key:

- MedicalRecordID

This design allows multiple diagnoses to be associated with a single medical record.

Prescription Table

The Prescription table stores prescribed medications.

Primary Key:

- PrescriptionID

Foreign Key:

- MedicalRecordID

This design allows multiple medications to be prescribed during a single visit.

PatientAllergy Table

The PatientAllergy table stores patient allergies.

Composite Primary Key:

- PatientID
- AllergyName

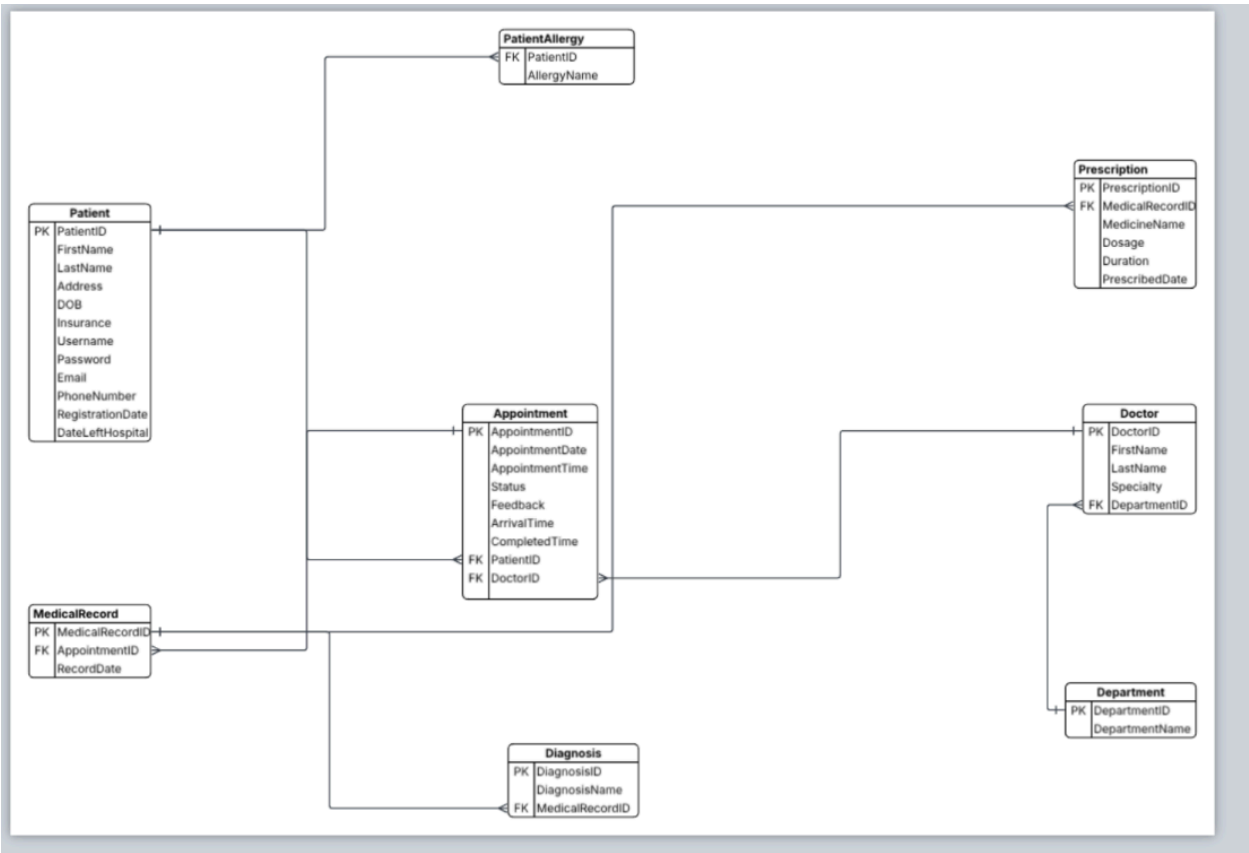
Foreign Key:

- PatientID

This structure allows a patient to have multiple allergies while preventing duplicate allergy entries.

1.3 Entity Relationship Diagram (ERD)

The final database design is illustrated in the photo below. ERD shows the tables created, their primary keys, foreign keys and the relationships between entities.



The Patient table acts as one of the central entities within the database. Each patient can have multiple appointments and multiple recorded allergies. This relationship is represented through the Appointment and PatientAllergy tables.

The Appointment table connects patients and doctors. Each appointment is associated with one patient and one doctor, while both patients and doctors can have multiple appointments over time.

The MedicalRecord table stores clinical information generated from appointments. A one-to-one relationship was implemented between Appointment and MedicalRecord by applying a unique constraint to AppointmentID within the MedicalRecord table. This ensures that a single appointment can only generate one medical record.

The Diagnosis and Prescription tables were separated from MedicalRecord to support situations where multiple diagnoses or medications may be recorded during a single consultation. This design prevents repeating groups and supports First Normal Form.

The Doctor table stores doctor information and is linked to the Department table through a foreign key relationship. This allows multiple doctors to belong to the same department while ensuring department information is maintained in a single location.

Overall, the relationships implemented support the hospital's operational requirements while maintaining referential integrity and reducing data redundancy.

1.4 Data Types and Design Decisions

Appropriate data types were selected for each attribute based on the nature of the data being stored.

Integer Data Types

INT was used for all primary key and foreign key fields, including PatientID, DoctorID, AppointmentID and DepartmentID. This data type was selected because it provides efficient storage and fast indexing for relational joins.

IDENTITY columns were used for primary keys to ensure that unique identifiers are generated automatically.

Character Data Types

VARCHAR was used for text-based attributes such as names, addresses, insurance providers, diagnoses and medication names.

Different lengths were selected depending on the expected size of the data. For example, FirstName and LastName were limited to 50 characters, while Address was allocated 150 characters to accommodate longer addresses.

Date Data Types

DATE was used for attributes such as DateOfBirth, RegistrationDate, AppointmentDate and PrescribedDate because time information is not required for these values.

Time Data Types

TIME was used for AppointmentTime, ArrivalTime and CompletedTime because only the time component is required.

Password Storage

The database stores a PasswordHash field rather than a plain-text password. This approach follows security best practices by preventing sensitive credentials from being stored directly within the database.

1.5 Constraints and Data Integrity

Several constraints were implemented to improve data quality and enforce business rules.

Primary Keys

Primary keys were implemented on all tables to uniquely identify records.

Examples include:

- PatientID
- DoctorID
- AppointmentID
- MedicalRecordID
- PrescriptionID

Foreign Keys

Foreign keys were implemented to maintain referential integrity between related tables.

Examples include:

- Appointment.PatientID → Patient.PatientID
- Appointment.DoctorID → Doctor.DoctorID
- Doctor.DepartmentID → Department.DepartmentID
- Diagnosis.MedicalRecordID → MedicalRecord.MedicalRecordID
- Prescription.MedicalRecordID → MedicalRecord.MedicalRecordID

These constraints prevent orphaned records from being created.

Unique Constraints

Unique constraints were used to prevent duplicate values.

Examples include:

- Username in the Patient table.
- DepartmentName in the Department table.
- Doctor appointment scheduling through:
- UNIQUE (DoctorID, AppointmentDate, AppointmentTime)
- Patient appointment scheduling through:
- UNIQUE (PatientID, AppointmentDate, AppointmentTime)

These constraints help prevent duplicate bookings.

Check Constraints

Check constraints were implemented to enforce valid business rules.

Examples include:

- Date of birth cannot be in the future.
- Registration date cannot be in the future.
- Appointment dates cannot be booked in the past.
- Appointment status is limited to valid values.
- CompletedTime must be later than ArrivalTime.

These constraints help ensure that invalid data cannot be inserted into the database.

1.6 Data Population

After creating the database structure, sample data was inserted into all tables to support testing and validation.

A minimum of seven records was inserted into each of the core entities, including patients, doctors, appointments and medical records. Additional records were inserted into the Diagnosis, Prescription and PatientAllergy tables to support testing of queries, stored procedures, views and triggers.

```
INSERT INTO Doctor
(
FirstName,
LastName,
Specialty,
DepartmentID
)
VALUES
('John','Smith','Cardiologist',1),
('Sarah','Johnson','Neurologist',2),
('Michael','Brown','Orthopedic Surgeon',3),
('Emma','Davis','Pediatrician',4),
('David','Wilson','Dermatologist',5),
('Sophia','Taylor','Oncologist',6),
('James','Anderson','General Surgeon',7);
```

```
INSERT INTO Patient
(
FirstName,
LastName,
Address,
DateOfBirth,
Insurance,
Username,
PasswordHash,
Email,
PhoneNumber,
RegistrationDate
)
VALUES
('Alice','Walker','12 Main Street','1990-05-15','AXA','alice90','HASH001','alice@email.com','1234567890','2025-01-01'),

('Robert','King','15 Oak Road','1985-08-22','Allianz','rob85','HASH002','robert@email.com','1234567891','2025-01-05'),

('Linda','Scott','20 Pine Lane','1995-03-10','AXA','linda95','HASH003','linda@email.com','1234567892','2025-01-10'),

('Mark','Evans','7 River Ave','1988-07-18','Bupa','mark88','HASH004','mark@email.com','1234567893','2025-01-12'),

('Nancy','Young','40 Green St','1992-12-01','Allianz','nancy92','HASH005','nancy@email.com','1234567894','2025-01-15'),

('Peter','Hall','55 King Rd','1980-06-06','Bupa','peter80','HASH006','peter@email.com','1234567895','2025-01-20'),

('Olivia','White','3 Queen Ave','2000-09-09','AXA','olivia00','HASH007','olivia@email.com','1234567896','2025-01-25');
```

	PatientID	FirstName	LastName	Address	DateOfBirth	Insurance	Username	Email	PhoneNumber	RegistrationDate	DateLeftHospital	PasswordHash
1	1	Alice	Walker	12 Main Street	1990-05-15	AXA	alice90	alice@email.com	1234567890	2025-01-01	NULL	HASH001
2	2	Robert	King	15 Oak Road	1985-08-22	Allianz	rob85	robert@email.com	1234567891	2025-01-05	NULL	HASH002
3	3	Linda	Scott	20 Pine Lane	1995-03-10	AXA	linda95	linda@email.com	1234567892	2025-01-10	NULL	HASH003
4	4	Mark	Evans	7 River Ave	1988-07-18	Bupa	mark88	mark@email.com	1234567893	2025-01-12	NULL	HASH004
5	5	Nancy	Young	40 Green St	1992-12-01	Allianz	nancy92	nancy@email.com	1234567894	2025-01-15	NULL	HASH005
6	6	Peter	Hall	55 King Rd	1980-06-06	Bupa	peter80	peter@email.com	1234567895	2025-01-20	NULL	HASH006
7	7	Olivia	White	3 Queen Ave	2000-09-09	AXA	olivia00	olivia@email.com	1234567896	2025-01-25	NULL	HASH007

Part 2
Question 2: Appointment Date Constraint

One of the business requirements specified that appointments should not be booked in the past. To enforce this rule at the database level, a CHECK constraint was implemented within the Appointment table.

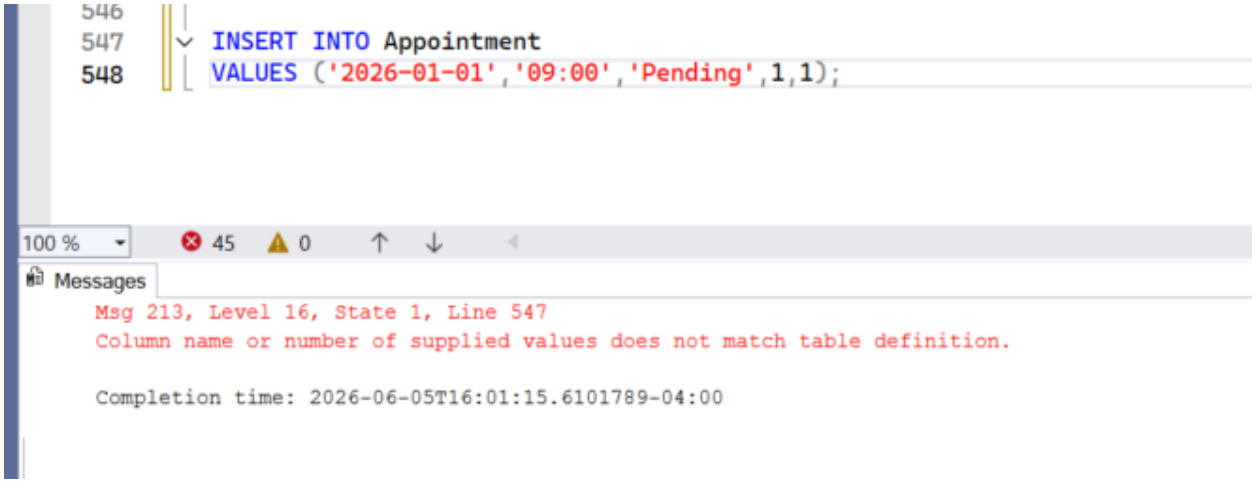
The constraint validates the AppointmentDate field whenever a new appointment is inserted or an existing appointment is updated. If a user attempts to enter a date earlier than the current system date, SQL Server rejects the operation and returns an error.

The following constraint was added to the AppointmentDate column:

```
## 1.4 APPOINTMENT TABLE

CREATE TABLE Appointment (
AppointmentID INT IDENTITY(1,1) PRIMARY KEY,
AppointmentDate DATE NOT NULL
CHECK (AppointmentDate >= CAST(GETDATE() AS DATE)),
AppointmentTime TIME NOT NULL,
Status VARCHAR(20) NOT NULL
CHECK (Status IN ('Pending', 'Cancelled', 'Completed', 'Available')),
Feedback VARCHAR(500),
ArrivalTime TIME,
CompletedTime TIME,
CHECK (CompletedTime > ArrivalTime),
PatientID INT NOT NULL,
FOREIGN KEY (PatientID)
REFERENCES Patient(PatientID),
DoctorID INT NOT NULL,
FOREIGN KEY (DoctorID)
REFERENCES Doctor(DoctorID),
UNIQUE (DoctorID, AppointmentDate, AppointmentTime),
UNIQUE (PatientID, AppointmentDate, AppointmentTime)
);
```

To test the constraint, an appointment was inserted using a date in the past. SQL Server correctly prevented the record from being created and displayed an error message.



Question 3: Patients Older Than 40 with a Cancer Diagnosis

To support this requirement, a user-defined function named CalculateAge was created. The function calculates a patient's age based on their date of birth and the current system date.

```
CREATE FUNCTION dbo.CalculateAge
(
@DateOfBirth DATE
)
RETURNS INT
AS
BEGIN
DECLARE @Age INT;

SET @Age = DATEDIFF(YEAR, @DateOfBirth, GETDATE());

RETURN @Age;

END;
GO
```

The function was then incorporated into a query that joins the Patient, Appointment, MedicalRecord and Diagnosis tables. These joins were required because diagnosis information is stored separately from patient information as part of the normalized

database design.

```sql
SELECT DISTINCT
    p.PatientID,
    p.FirstName,
    p.LastName,
    dbo.CalculateAge(p.DateOfBirth) AS Age,
    d.DiagnosisName
FROM Patient p

JOIN Appointment a
    ON p.PatientID = a.PatientID

JOIN MedicalRecord m
    ON a.AppointmentID = m.AppointmentID

JOIN Diagnosis d
    ON m.MedicalRecordID = d.MedicalRecordID

WHERE dbo.CalculateAge(p.DateOfBirth) > 40
AND d.DiagnosisName = 'Cancer Screening';
```

The query filters patients whose age is greater than 40 years and whose diagnosis is recorded as Cancer Screening.

	PatientID	FirstName	LastName	Age	DiagnosisName
1	6	Peter	Hall	46	Cancer Screening

Question 4a: Search Medicines by Name

To satisfy this requirement, a stored procedure named SearchMedicineByName was created. The procedure accepts a search term as a parameter and searches the Prescription table using the LIKE operator. This allows users to search using either full medicine names or partial text values.

The results are sorted in descending order of PrescribedDate so that the most recently prescribed medicines appear first.

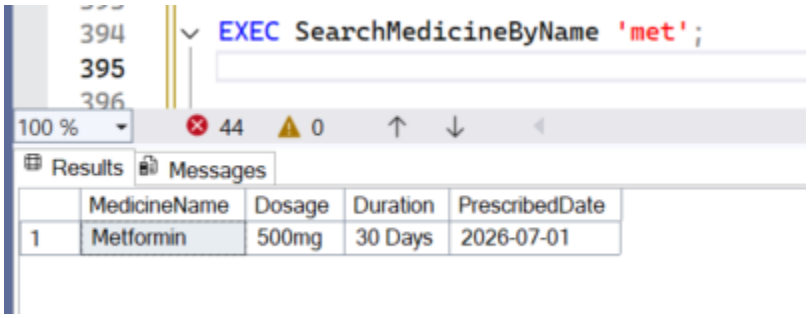
```
CREATE PROCEDURE SearchMedicineByName
@SearchTerm VARCHAR(50)
AS
BEGIN

SELECT
    MedicineName,
    Dosage,
    Duration,
    PrescribedDate
FROM Prescription
WHERE MedicineName LIKE '%' + @SearchTerm + '%'
ORDER BY Prescribe
```

column MedicineName(varchar, not null)

```
END;
GO
```

The procedure was executed using the search term "met".



The output successfully returned all matching medicines ordered from the most recent prescription date to the oldest.

Question 4b: Return Diagnosis and Allergies for a Patient with an Appointment Today

To achieve this requirement, a stored procedure named GetPatientDiagnosisAndAllergies was created. The procedure accepts a PatientID as a parameter and retrieves information from multiple related tables.

The first query returns diagnosis information by joining the Appointment, MedicalRecord and Diagnosis tables. The second query returns allergy information from the PatientAllergy table.

The procedure uses the GETDATE() system function to ensure that only patients with appointments scheduled for the current system date are included in the results.

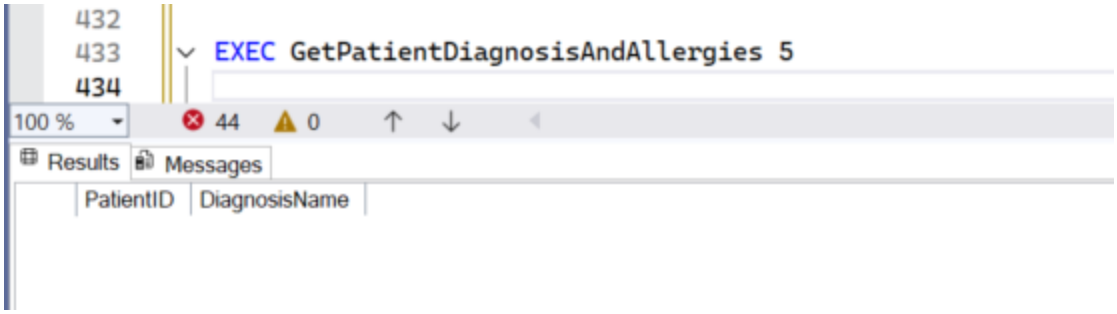
```
CREATE PROCEDURE GetPatientDiagnosisAndAllergies
@PatientID INT
AS
BEGIN


SELECT
    a.PatientID,
    d.DiagnosisName
FROM Appointment a
JOIN MedicalRecord m
    ON m.AppointmentID = a.AppointmentID
JOIN Diagnosis d
    ON m.MedicalRecordID = d.MedicalRecordID
WHERE a.AppointmentDate = CAST(GETDATE() AS DATE)
  AND a.PatientID = @PatientID;

SELECT
    p.PatientID,
    p.AllergyName
FROM Appointment a
JOIN PatientAllergy p
    ON a.PatientID = p.PatientID
WHERE a.AppointmentDate = CAST(GETDATE() AS DATE)
  AND p.PatientID = @PatientID;

END;
GO
```

The procedure was executed using a valid PatientID.



Question 4c: Update Existing Doctor Details

The hospital requested a mechanism for updating doctor information without manually writing update statements each time a change is required.

A stored procedure named UpdateDoctorDetails was created to support this functionality. The procedure accepts parameters for the doctor's first name, last name, specialty, department and doctor identifier.

```
CREATE PROCEDURE UpdateDoctorDetails
@NewDoctorFirstName VARCHAR(100),
@NewDoctorLastName VARCHAR(100),
@NewDoctorSpecialty VARCHAR(50),
@NewDoctorDepartmentID INT,
@DoctorID INT
AS
BEGIN

UPDATE Doctor
SET FirstName = @NewDoctorFirstName,
    LastName = @NewDoctorLastName,
    Specialty = @NewDoctorSpecialty,
    DepartmentID = @NewDoctorDepartmentID
WHERE DoctorID = @DoctorID;


END;
GO
```

The procedure was tested by updating an existing doctor's details.

To verify that the update was successful, the doctor record was queried before and after the procedure was executed.

```
460  SELECT *
461  FROM Doctor
```

100 % 43 0

Results Messages

	DoctorID	FirstName	LastName	Specialty	DepartmentID
1	1	Ugo	Ikem	General Surgeon	7
2	2	Sarah	Johnson	Neurologist	2
3	3	Michael	Brown	Orthopedic Surgeon	3
4	4	Emma	Davis	Pediatrician	4
5	5	David	Wilson	Dermatologist	5
6	6	Sophia	Taylor	Oncologist	6
7	7	James	Anderson	General Surgeon	7

```
460  EXEC UpdateDoctorDetails 'Isaac', 'Nana', 'Cardiologist', 1, 7;
461  SELECT *
462  FROM Doctor;
463
```

100 % 43 0

Results Messages

	DoctorID	FirstName	LastName	Specialty	DepartmentID
1	1	Ugo	Ikem	General Surgeon	7
2	2	Sarah	Johnson	Neurologist	2
3	3	Michael	Brown	Orthopedic Surgeon	3
4	4	Emma	Davis	Pediatrician	4
5	5	David	Wilson	Dermatologist	5
6	6	Sophia	Taylor	Oncologist	6
7	7	Isaac	Nana	Cardiologist	1

The results confirmed that the doctor's information was successfully modified within the database.

Question 4d: Delete Completed Appointments

To meet this requirement, a stored procedure named DeleteCompleteStatus was created. The procedure accepts a status value as a parameter and removes matching records from the Appointment table.

```sql
CREATE PROCEDURE DeleteCompleteStatus
@CompletedStatus VARCHAR(50)
AS
BEGIN


DELETE FROM Appointment
WHERE Status = @CompletedStatus;


END;
GO
```

Question 5: Appointment Details View

To satisfy this requirement, a view named AppointmentDetails was created. The view combines information from the Appointment, Doctor and Department tables using inner joins.

```sql
CREATE VIEW AppointmentDetails AS

SELECT
d.DoctorID,
d.FirstName,
d.LastName,
d.Specialty,
de.DepartmentName,
a.AppointmentDate,
a.AppointmentTime,
a.Feedback
FROM Appointment a
JOIN Doctor d
ON a.DoctorID = d.DoctorID
JOIN Department de
ON d.DepartmentID = de.DepartmentID;
GO
```

The view was then queried using a SELECT statement.

```
503  SELECT *
504  FROM AppointmentDetails;
505  ---
```

100 % 42 0

Results Messages

	DoctorID	FirstName	LastName	Specialty	DepartmentName	AppointmentDate	AppointmentTime	Feedback
1	1	Ugo	Ikem	General Surgeon	General Surgery	2026-07-01	09:00:00.0000000	NULL
2	1	Ugo	Ikem	General Surgeon	General Surgery	2026-07-01	10:00:00.0000000	NULL
3	2	Sarah	Johnson	Neurologist	Neurology	2026-07-02	11:00:00.0000000	NULL
4	3	Michael	Brown	Orthopedic Surgeon	Orthopedics	2026-07-02	12:00:00.0000000	NULL
5	4	Emma	Davis	Pediatrician	Pediatrics	2026-07-03	09:30:00.0000000	NULL
6	5	David	Wilson	Dermatologist	Dermatology	2026-07-03	10:30:00.0000000	NULL
7	6	Sophia	Taylor	Oncologist	Oncology	2026-07-04	14:00:00.0000000	NULL

Question 6: Appointment Status Trigger

To implement this requirement, an AFTER UPDATE trigger named ChangeStatusWhenCancelled was created on the Appointment table. The trigger monitors updates made to appointment records and checks whether the status has been changed to "Cancelled".

If a cancelled appointment is detected, the trigger automatically updates the appointment status to "Available". This allows the appointment slot to become available for future bookings without requiring manual intervention by hospital staff.

```
CREATE TRIGGER ChangeStatusWhenCancelled
ON Appointment
AFTER UPDATE
AS
BEGIN

UPDATE Appointment
SET Status = 'Available'
WHERE AppointmentID IN
(
    SELECT AppointmentID
    FROM inserted
    WHERE Status = 'Cancelled'
);


END;
GO
```

Before

```
528
529  SELECT *
530  FROM Appointment
531  ---
```

100 % 43 0

Results Messages

	AppointmentID	AppointmentDate	AppointmentTime	Status	Feedback	ArrivalTime	CompletedTime	PatientID	DoctorID
1	1	2026-07-01	09:00:00.0000000	Pending	NULL	NULL	NULL	1	1
2	2	2026-07-01	10:00:00.0000000	Pending	NULL	NULL	NULL	2	1
3	3	2026-07-02	11:00:00.0000000	Pending	NULL	NULL	NULL	3	2
4	4	2026-07-02	12:00:00.0000000	Pending	NULL	NULL	NULL	4	3
5	5	2026-07-03	09:30:00.0000000	Pending	NULL	NULL	NULL	5	4
6	6	2026-07-03	10:30:00.0000000	Pending	NULL	NULL	NULL	6	5
7	7	2026-07-04	14:00:00.0000000	Pending	NULL	NULL	NULL	7	6

The trigger was tested by updating an appointment status to "Cancelled".

```
529  UPDATE Appointment
530  SET Status = 'Cancelled'
531  WHERE AppointmentID = 2;
532  SELECT *
533  FROM Appointment
534  ---
```

100 % 43 0

Results Messages

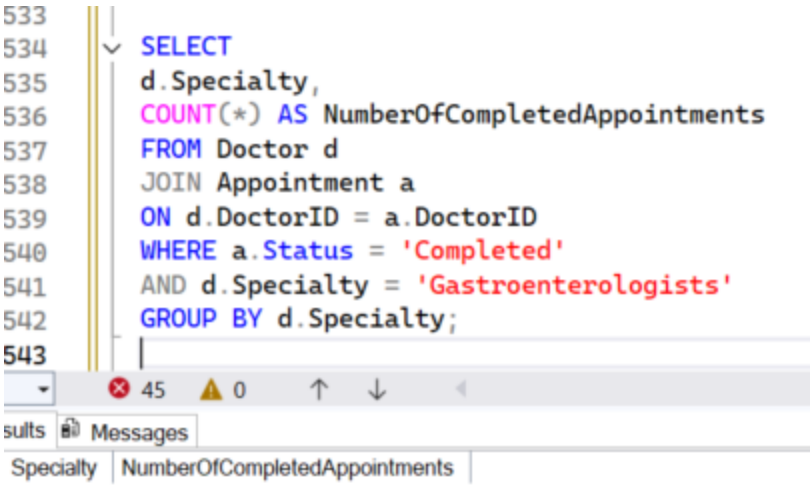
	AppointmentID	AppointmentDate	AppointmentTime	Status	Feedback	ArrivalTime	CompletedTime	PatientID	DoctorID
1	1	2026-07-01	09:00:00.0000000	Pending	NULL	NULL	NULL	1	1
2	2	2026-07-01	10:00:00.0000000	Available	NULL	NULL	NULL	2	1
3	3	2026-07-02	11:00:00.0000000	Pending	NULL	NULL	NULL	3	2
4	4	2026-07-02	12:00:00.0000000	Pending	NULL	NULL	NULL	4	3
5	5	2026-07-03	09:30:00.0000000	Pending	NULL	NULL	NULL	5	4
6	6	2026-07-03	10:30:00.0000000	Pending	NULL	NULL	NULL	6	5
7	7	2026-07-04	14:00:00.0000000	Pending	NULL	NULL	NULL	7	6

The results confirmed that the trigger executed successfully and changed the appointment status to "Available".

Question 7: Number of Completed Appointments for Gastroenterologists

To achieve this requirement, a query was created that joins the Doctor and Appointment tables and counts appointments where the status is recorded as "Completed".

The query filters records using the doctor's specialty and groups the results by specialty.



Data Integrity and Concurrency

Data integrity was a key consideration throughout the design of the hospital database. Several measures were implemented to ensure that accurate and consistent information is maintained.

Primary keys were used on all tables to uniquely identify records. Foreign key constraints were implemented to ensure that relationships between tables remain valid. For example, an appointment cannot reference a patient or doctor that does not exist within the database.

Check constraints were used to enforce business rules. Examples include preventing appointment dates from being entered in the past, ensuring valid appointment status values and preventing invalid date combinations.

Unique constraints were also used to prevent duplicate appointment bookings. A doctor cannot be booked for multiple appointments at the same date and time, and a patient cannot create duplicate appointments for the same time slot.

Concurrency is particularly important in a hospital environment where multiple users may be accessing the database simultaneously. Reception staff may be booking appointments while doctors are updating medical records and patients are accessing the patient portal.

SQL Server manages concurrency through transaction management and locking mechanisms. In this database, the use of primary keys, foreign keys and unique constraints helps reduce the likelihood of conflicting updates and duplicate records being created during concurrent operations.

Database Security

Patient information represents highly sensitive data and therefore security was considered during the database design process.

The Patient table stores password hashes rather than plain-text passwords. This reduces the risk of credential exposure if unauthorized access to the database occurs.

Access to the database should be controlled through SQL Server authentication and role-based permissions. Different users should have different levels of access depending on their responsibilities.

For example:

- Reception staff should be permitted to manage appointments but not modify medical records.
- Doctors should be able to access patient medical information and update diagnoses and prescriptions.
- Database administrators should be responsible for database maintenance and security management.
- Patients should only be able to access their own information through the patient portal.

Additional security measures such as encryption, strong password policies and audit logging should also be implemented in a production environment.

Database Backup and Recovery

The hospital relies on the availability of patient and appointment information to deliver healthcare services. As a result, an effective backup and recovery strategy is essential.

A full database backup should be performed regularly to ensure that all patient records, appointments, diagnoses and prescriptions can be restored if a failure occurs.

In addition to full backups, differential backups and transaction log backups should be implemented to minimise potential data loss between backup cycles.

Backups should be stored securely in a separate location from the production database to protect against hardware failure, accidental deletion or ransomware attacks.

Recovery procedures should be tested periodically to ensure that backups can be restored successfully when required. This helps guarantee that critical hospital operations can continue following an unexpected system failure.

Conclusion

This project successfully designed and implemented a hospital database system using Microsoft SQL Server and T-SQL. The database was normalised to Third Normal Form (3NF) to minimise redundancy and improve data consistency.

The solution includes tables, constraints, stored procedures, a user-defined function, a view and a trigger to satisfy the functional requirements provided by the hospital.

The implemented design supports patient registration, appointment scheduling, medical record management, diagnosis tracking, prescription management and allergy recording while maintaining data integrity and security.

Overall, the final solution provides a scalable and maintainable database structure capable of supporting the operational requirements of the hospital.